

How to create colour gradient in Python?

Asked 10 years ago Modified 1 year, 5 months ago Viewed 187k times



76



Highly active question. You have enough reputation to answer or unprotect this question.

I want to create a new colormap which interpolates between green and blue (or any other two colours for that matter). My goal is to get something like:



First of all I am really not sure if this can be done using linear interpolation of blue and green. If it can, I'm not sure how to do so, I found some documentation on using a matplotlib method that interpolates specified RGB values [here](#)

The real trouble is understanding how "cdict2" works below. For the example the documentation says:

"Example: suppose you want red to increase from 0 to 1 over the bottom half, green to do the same over the middle half, and blue over the top half. Then you would use:"

```
from matplotlib import pyplot as plt
import matplotlib
import numpy as np

plt.figure()
a=np.outer(np.arange(0,1,0.01),np.ones(10))
cdict2 = {'red':  [(0.0,  0.0, 0.0),
                  (0.5,  1.0, 1.0),
                  (1.0,  1.0, 1.0)],
          'green': [(0.0,  0.0, 0.0),
                   (0.25, 0.0, 0.0),
                   (0.75, 1.0, 1.0),
                   (1.0,  1.0, 1.0)],
          'blue':  [(0.0,  0.0, 0.0),
                   (0.5,  0.0, 0.0),
                   (1.0,  1.0, 1.0)]}

my_cmap2 = matplotlib.colors.LinearSegmentedColormap('my_colormap2',cdict2,256)
plt.imshow(a,aspect='auto', cmap =my_cmap2)
plt.show()
```

EDIT: I now understand how the interpolation works, for example this will give a red to white interpolation:

White to red: Going down the columns of the "matrix" for each colour, in column one we have the xcoordinate of where we want the interpolation to start and end and the two other columns are the actual values for the colour value at that coordinate.

```
cdict2 = {'red':  [(0.0,  1.0, 1.0),
                  (1.0,  1.0, 1.0),
                  (1.0,  1.0, 1.0)],
          'green': [(0.0,  1.0, 1.0),
                   (1.0,  0.0, 0.0),
                   (1.0,  0.0, 0.0)],
          'blue':  [(0.0,  1.0, 1.0),
                   (1.0,  0.0, 0.0),
                   (1.0,  0.0, 0.0)]}
```

It is evident that the gradient I want will be very difficult to create by interpolating in RGB space...

python

colors

gradient

Edit tags

Share Edit Follow Close Flag Unprotect

edited Sep 4, 2014 at 15:33

asked Sep 4, 2014 at 15:01

Dipole

1,900 2 25 37

- [Check out this link](#) about the named colors. There's code in there that shows conversion between the specification methods. [I also think this link](#) about colorbars might help. – [mauve](#) Sep 4, 2014 at 15:15
- 1 How did you create that example gradient? It's far from linear. – [Mark Ransom](#) Sep 4, 2014 at 18:04
- 1 Yes absolutely, its just a screen shot illustrating what I want. I didn't create it. Im wondering if Python has some functions which facilitate those kinds of gradients... – [Dipole](#) Sep 4, 2014 at 18:10
- 2 A screen shot from *what* though? – [Mark Ransom](#) Sep 4, 2014 at 18:32
- I can try to find the slide I got it from, if that will help, but I just remember it being something that said "Here is an example of a colorgradient" – [Dipole](#) Sep 4, 2014 at 21:44

11 Answers

Sorted by:

Reset to default

Date modified (newest first)



0



A variation of J.Zhao's post which hopefully simplifies what is happening

```
def generateGradientBackground(w_ori, h_ori, r, g, b, brightness=1,
reverse=False):

    # generate a vertical gradient between a colour and white
    # create the 1st column 1 pixel wide with all the colour ranges needed
    # then copy this horizontally for as many pixels width needed

    if reverse:
        s = (255*brightness,255*brightness,255*brightness)
        e = (r*brightness,g*brightness,b*brightness)
    else:
        s = (r*brightness,g*brightness,b*brightness)
        e = (255*brightness,255*brightness,255*brightness)

    #transition per line
    r = (s[0] - e[0])/(h_ori-1)
```

```
g = (s[1] - e[1])/(h_ori-1)
b = (s[2] - e[2])/(h_ori-1)

t = ()

for h in range(h_ori):
    # get new pixel colour
    p = (
        int(max((s[0] - r*h),0)),
        int(max((s[1] - g*h),0)),
        int(max((s[2] - b*h),0))
    )

    hlist.append(p)

# now we have all the pixels for 1 column

for p in hlist:
    tp = ( p,) * w_ori )
    pixel_list += list( tp )

im = Image.new('RGB', (w_ori,h_ori))
im.putdata(pixel_list)
#im.show()
#im.save("background.png")
return im
```

Share Edit Follow Flag

answered Mar 21, 2023 at 17:48

 Hebbs

17 2

For what is worth, here is a neat numpy way of interpolating between two colors:

0

```
def h_kernel(shape):
    row = np.linspace(0, 1, shape[1])
    kernel_1d = np.tile(row, (shape[0], 1))
    kernel_3d = cv2.merge((kernel_1d, kernel_1d, kernel_1d))
    return kernel_3d

def v_kernel(shape):
    kernel = h_kernel((shape[1], shape[0], shape[2]))
    return cv2.rotate(kernel, cv2.cv2.ROTATE_90_CLOCKWISE)

def gradient(shape, c1, c2, kernel_func):
    im = np.zeros(shape)
    kernel = h_kernel(shape)
    im = kernel * c1 + (1 - kernel) * c2
    return im.astype(np.uint8)

shape = (540, 960)
c1 = np.array((182, 132, 69)) # bgr
c2 = np.array((47, 32, 9))
h_gradient = gradient(shape, c1, c2, h_kernel)
v_gradient = gradient(shape, c1, c2, v_kernel)
cv2.imwrite("h.png", h_gradient)
cv2.imwrite("v.png", v_gradient)
```





Share Edit Follow Flag

edited Jun 26, 2022 at 18:39

answered Apr 2, 2022 at 2:19

 elbashmubarmeg

374 1 10

-  why vertical one is not working !!?? – MSI Jun 26, 2022 at 7:19
-  Oops, the arguments to the imshow function were incomplete. I fixed it. – elbashmubarmeg Jun 26, 2022 at 18:40

 maybe this [link](#) could help you solve your problem, thanks this code's author **redjack001**

0







```
from PIL import Image

#set the size of the new image
w = 500
h = 200

#start creating gradient
pixel_list = []
pixel_w = []

#colour at 0,0
start1 = (255,255,255)
s = list(start1)
pixel_list.append(start1)
print('Position zero:', pixel_list[0])

#colour at end
end1 = (174,15,15)
e = list(end1)

#in case you want to change the final colour you could use f to adjust otherwise
just keep it at 1
f = 1

#transition per line
r = (s[0] - e[0])/w*f
g = (s[1] - e[1])/w*f
b = (s[2] - e[2])/w*f

t = ()

for j in range (0,w):
    t = pixel_list[j]

    #convert pixel tuple to a list and recalculate
    li = list(t)
    li[0] = int(max((li[0] - r*j),0))
    li[1] = int(max((li[1] - g*j),0))
    li[2] = int(max((li[2] - b*j),0))
    z = (li[0],li[1],li[2])
    final_t = tuple(z)
    #print('final_t:', final_t) if you want to show the list of pixel values
    pixel_list[j] = final_t
    for i in range (0,h):
        pixel_w = []
        pixel_w.append(final_t)
        pixel_list.extend(pixel_w)

l = len(pixel_list)

print('Pixel_list:', pixel_list, l)

#remove the last item
del pixel_list[l-1:]

print('Reprint length:', len(pixel_list))

im = Image.new('RGB', (w,h))
im.putdata(pixel_list)
im.show()
```

the result is here:



Share Edit Follow Flag

edited Nov 30, 2021 at 9:48

answered Nov 30, 2021 at 7:45

 J.Zhao
2,310 1 15 14



I needed this as well, but I wanted to enter multiple arbitrary color points. Consider a heat map where you need black, blue, green... all the way up to "hot" colors. I borrowed [Mark Ransom](#)'s code above and extended it to meet my needs. I'm very happy with it. My thanks to all, especially Mark.

12



This code is neutral to the size of the image (no constants in the gaussian distribution); you can change it with the width= parameter to pixel(). It also allows tuning the "spread" (-> stddev) of the distribution; you can muddle them up further or introduce black bands by changing the spread= parameter to pixel().



```
#!/usr/bin/env python

width, height = 1000, 200

import math
from PIL import Image
im = Image.new('RGB', (width, height))
ld = im.load()

# A map of rgb points in your distribution
# [distance, (r, g, b)]
# distance is percentage from left edge
heatmap = [
    [0.0, (0, 0, 0)],
    [0.20, (0, 0, .5)],
    [0.40, (0, .5, 0)],
    [0.60, (.5, 0, 0)],
    [0.80, (.75, .75, 0)],
    [0.90, (1.0, .75, 0)],
    [1.00, (1.0, 1.0, 1.0)],
]

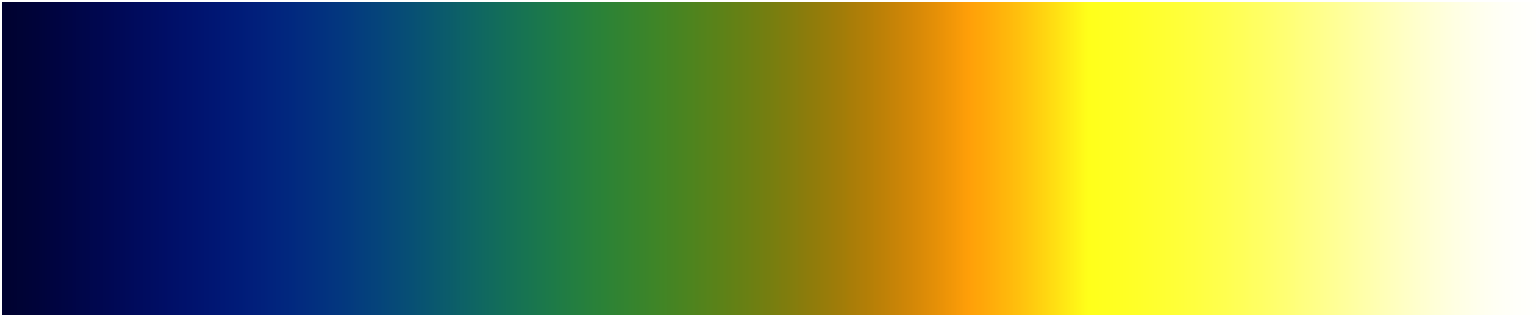
def gaussian(x, a, b, c, d=0):
    return a * math.exp(-(x - b)**2 / (2 * c**2)) + d

def pixel(x, width=100, map=[], spread=1):
    width = float(width)
    r = sum([gaussian(x, p[1][0], p[0] * width, width/(spread*len(map))) for p in map])
    g = sum([gaussian(x, p[1][1], p[0] * width, width/(spread*len(map))) for p in map])
    b = sum([gaussian(x, p[1][2], p[0] * width, width/(spread*len(map))) for p in map])
    return min(1.0, r), min(1.0, g), min(1.0, b)

for x in range(im.size[0]):
    r, g, b = pixel(x, width=im.size[0], map=heatmap)
    r, g, b = [int(256*v) for v in (r, g, b)]
    for y in range(im.size[1]):
        ld[x, y] = r, g, b

im.save('grad.png')
```

Here's the multi-point gradient produced by this code:



Share Edit Follow Flag

edited May 25, 2021 at 1:04

answered Jun 29, 2015 at 20:47

 dgc
565 3 7



A simple answer I have not seen yet is to just use the [colour package](#).

138

Install via pip



pip install colour



Use as so:

```
from colour import Color
red = Color("red")
colors = list(red.range_to(Color("green"),10))

# colors is now a list of length 10
# Containing:
# [<Color red>, <Color #f13600>, <Color #e36500>, <Color #d58e00>, <Color #c7b000>, <Color #a4b800>, <Color #72aa00>, <Color #459c00>, <Color #208e00>, <Color green>]
```

Change the inputs to any colors you want. As noted by @zelusp, this will *not* restrict itself to a smooth combination of only two colors (e.g. red to blue will have yellow+green in the middle), but based on the upvotes it's clear a number of folks find this to be a useful approximation

Share Edit Follow Flag

edited Jan 19, 2021 at 14:19

answered Apr 19, 2015 at 21:22

 Nico Schlömer

57.9k33211275

 Hamy

21.3k1679104

- 5

▲

▼

Gradients produced by this method pass through other colors to form a gradient - It's not a true gradient of only two colors - i.e. using this to go from red to blue will generate yellow and green colors. – zelusp Feb 28, 2019 at 23:39
- 10

▲

▼

[This page](#) is a phenomenal resource about color gradation in Python – zelusp Mar 25, 2019 at 20:14
- ▲

▼

at least the package name is spelt correctly compared with the example;p – Hayden Thring Sep 8, 2020 at 1:43
- ▲

▼

This is a HSV gradient. The author could specify that in either the answer, or his code. Otherwise, good job. – Captain Trojan Sep 11, 2020 at 18:18

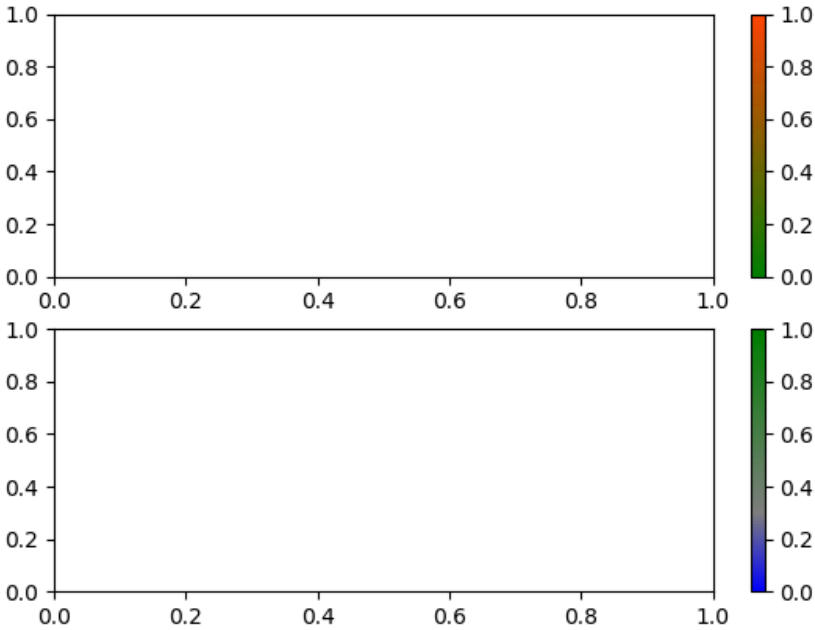
▲

▼

10

🔖

🕒



This is a very compact way to create a colormap. See also the documentation of [LinearSegmentedColormap](#).

```
import matplotlib as mpl
import matplotlib.pyplot as plt

cmap0 = mpl.colors.LinearSegmentedColormap.from_list(
    'green2red', ['green', 'orangered'])
cmap1 = mpl.colors.LinearSegmentedColormap.from_list(
    'unevenly divided', [(0, 'b'), (.3, 'gray'), (1, 'green')])

# plot
fig, axs = plt.subplots(2, 1)
norm = mpl.colors.Normalize(vmin=0, vmax=1)
cbar = axs[0].figure.colorbar(
    mpl.cm.ScalarMappable(norm=norm, cmap=cmap0),
    ax=axs[0], fraction=.1)
cbar = axs[1].figure.colorbar(
    mpl.cm.ScalarMappable(norm=norm, cmap=cmap1),
    ax=axs[1], fraction=.1)
plt.show()
```

Share Edit Follow Flag

answered Apr 24, 2020 at 11:49

 Markus Dutschke

10.3k46963

- 1

▲

▼

This is one of the most simple ways. Underappreciated. `mpl.colors.LinearSegmentedColormap.from_list('b2g', ['royalblue', 'w', 'forestgreen'])` – H. Rev. Oct 17, 2022 at 18:18

▲

▼

60

🔖

🕒

If you just need to **interpolate in between 2 colors**, I wrote a simple function for that. `colorFader` creates you a hex color code out of two other hex color codes.

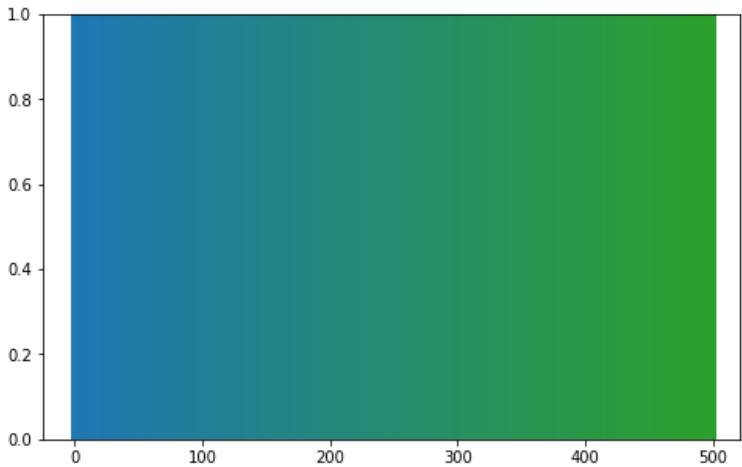
```
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np

def colorFader(c1,c2,mix=0): #fade (linear interpolate) from color c1 (at mix=0)
to c2 (mix=1)
    c1=np.array(mpl.colors.to_rgb(c1))
    c2=np.array(mpl.colors.to_rgb(c2))
    return mpl.colors.to_hex((1-mix)*c1 + mix*c2)

c1='#1f77b4' #blue
c2='green' #green
n=500

fig, ax = plt.subplots(figsize=(8, 5))
for x in range(n+1):
    ax.axvline(x, color=colorFader(c1,c2,x/n), linewidth=4)
plt.show()
```

result:



update due to high interest:

`colorFader` works now for rgb-colors and color-strings like 'red' or even 'r'.

Share Edit Follow Flag

edited Feb 21, 2019 at 15:05

answered Jun 10, 2018 at 13:07



Markus Dutschke

10.3k 4 69 63

- 1

this is failing for `c1='#ff0000'` #red and `c2='#001dff'` #blue – Parthiban Rajendran Aug 19, 2018 at 7:30
- This a great answer. Works for me if I substitute `hex(a)[2:]` with `hex(a)[2:4]` . – Ulf Aslak Aug 22, 2018 at 17:16
- 1

Yes! I actually miscommented above, I meant `hex(a)[2:-1]` . In my environment, `hex` returns a string that ends on "L", and you do not want to include that, hence the `-1` . – Ulf Aslak Aug 23, 2018 at 8:50
- 1

@Ulf Alsak: Completely updated. Should work on your system as well ;) – Markus Dutschke Jul 1, 2019 at 8:31
- 1

@Eddy axhline – Markus Dutschke Oct 8, 2022 at 7:17

|

8

The first element of each tuple (0, 0.25, 0.5, etc) is the place where the color should be a certain value. I took 5 samples to see the RGB components (in GIMP), and placed them in the tables. The RGB components go from 0 to 1, so I had to divide them by 255.0 to scale the normal 0-255 values.

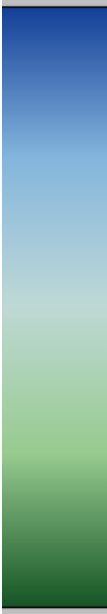
The 5 points are a rather coarse approximation - if you want a 'smoother' appearance, use more values.

```
from matplotlib import pyplot as plt
import matplotlib
import numpy as np

plt.figure()
a=np.outer(np.arange(0,1,0.01),np.ones(10))
fact = 1.0/255.0
cdict2 = {'red': [(0.0, 22*fact, 22*fact),
(0.25, 133*fact, 133*fact),
(0.5, 191*fact, 191*fact),
(0.75, 151*fact, 151*fact),
(1.0, 25*fact, 25*fact)],
'green': [(0.0, 65*fact, 65*fact),
(0.25, 182*fact, 182*fact),
(0.5, 217*fact, 217*fact),
(0.75, 203*fact, 203*fact),
(1.0, 88*fact, 88*fact)],
'blue': [(0.0, 153*fact, 153*fact),
(0.25, 222*fact, 222*fact),
(0.5, 214*fact, 214*fact),
(0.75, 143*fact, 143*fact),
(1.0, 40*fact, 40*fact)]}

my_cmap2 = matplotlib.colors.LinearSegmentedColormap('my_colormap2',cdict2,256)
plt.imshow(a,aspect='auto', cmap =my_cmap2)
plt.show()
```

Note that red is quite present. It's there because the center area approaches gray - where the three components are necessary.



This produces:

Share Edit Follow Flag

edited Apr 14, 2016 at 15:38

answered Sep 4, 2014 at 15:53



Gabriel

42.3k 80 240 424



jcoppens

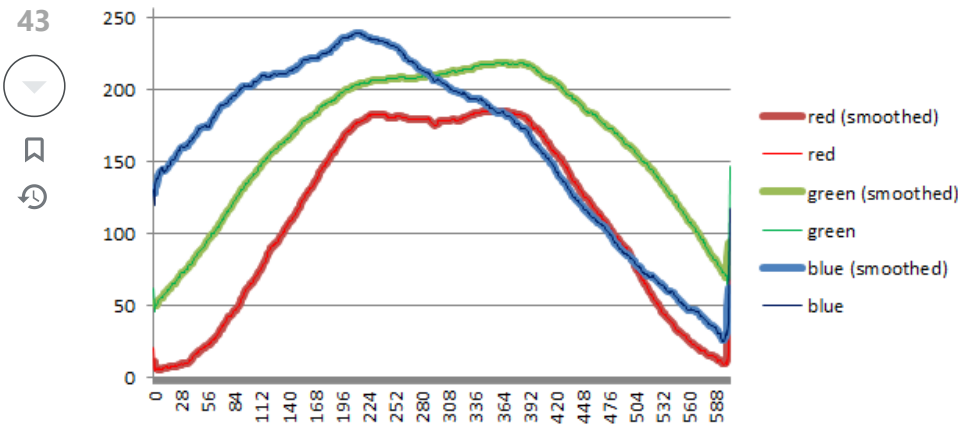
5,430 6 30 51

- 1

+1 for thinking outside the box, I never thought of actually sampling the components. Just out of curiosity, why does there need to be a 3rd column in the dictionary for each component, if the first value in each tuple is the position and the second is the value- what does the 3rd represent? – Dipole Sep 4, 2014 at 16:09

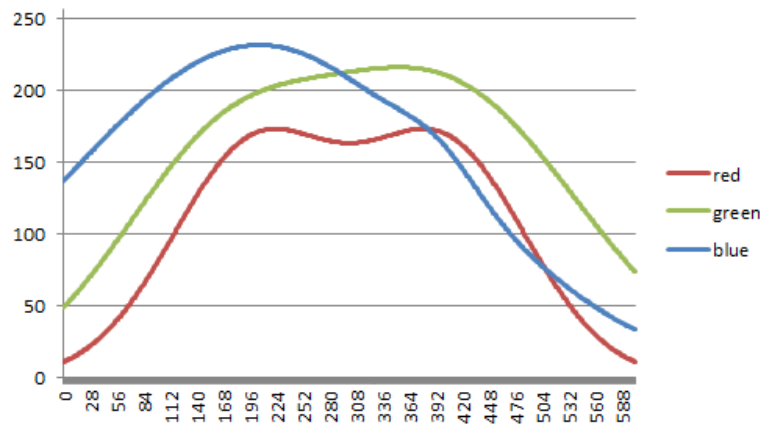
- @Jack: The two values allow for a discontinuity in the color map. If a point is $(x_0, y_{left}, y_{right})$, the colormap approaches y_{left} as x increases to x_0 , and it approaches y_{right} as x decreases to x_0 . – Warren Weckesser Sep 4, 2014 at 16:19
- Great, thanks a bunch! I have to admit, this was more complicated than I had previously imagined. Using your method I can get what I want provided I have a gradient at my disposal to get the component values of. But what if I want to create some the above but for red and green etc. I guess that won't be so straightforward. – Dipole Sep 4, 2014 at 16:43

It's obvious that your original example gradient is *not* linear. Have a look at a graph of the red, green, and blue values averaged across the image:



Attempting to recreate this with a combination of linear gradients is going to be difficult.

To me each color looks like the addition of two gaussian curves, so I did some best fits and came up with this:



Using these calculated values lets me create a really pretty gradient that matches yours almost exactly.

```
import math
from PIL import Image
im = Image.new('RGB', (604, 62))
ld = im.load()

def gaussian(x, a, b, c, d=0):
    return a * math.exp(-(x - b)**2 / (2 * c**2)) + d

for x in range(im.size[0]):
    r = int(gaussian(x, 158.8242, 201, 87.0739) + gaussian(x, 158.8242, 402, 87.0739))
    g = int(gaussian(x, 129.9851, 157.7571, 108.0298) + gaussian(x, 200.6831, 399.4535, 143.6828))
    b = int(gaussian(x, 231.3135, 206.4774, 201.5447) + gaussian(x, 17.1017, 395.8819, 39.3148))
    for y in range(im.size[1]):
        ld[x, y] = (r, g, b)
```



Unfortunately I don't yet know how to generalize it to arbitrary colors.

Share Edit Follow Flag

answered Sep 5, 2014 at 5:22
Mark Ransom
306k 44 413 642

- Thanks Mark, this is great. I have also been experimenting with different curves, but as you say I struggle to find any way to generalise this for arbitrary colours. Perhaps looking at how some of the standard python gradients created here wiki.scipy.org/Cookbook/Matplotlib/Show_colormaps would help, although I can't find code that shows how they were created. – Dipole Sep 5, 2014 at 11:52

This creates a colormap controlled by a single parameter, y :

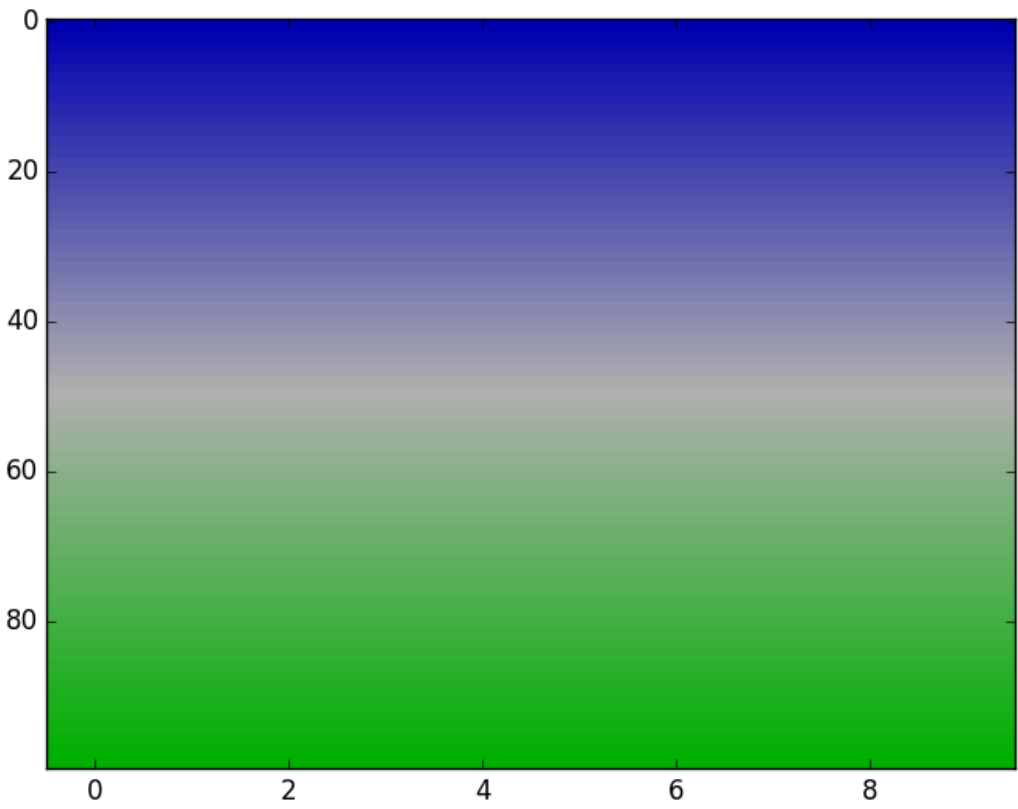
8

```
from matplotlib.colors import LinearSegmentedColormap

def bluegreen(y):
    red = [(0.0, 0.0, 0.0), (0.5, y, y), (1.0, 0.0, 0.0)]
    green = [(0.0, 0.0, 0.0), (0.5, y, y), (1.0, y, y)]
    blue = [(0.0, y, y), (0.5, y, y), (1.0, 0.0, 0.0)]
    colordict = dict(red=red, green=green, blue=blue)
    bluegreenmap = LinearSegmentedColormap('bluegreen', colordict, 256)
    return bluegreenmap
```

red ramps up from 0 to y and then back down to 0. green ramps up from 0 to y and then is constant. blue starts at y and is constant for the first half, then ramps down to 0.

Here's the plot with $y = 0.7$:



You could smooth it out by using adding another segment or two.

Share Edit Follow Flag

answered Sep 4, 2014 at 15:47
 **Warren Weckesser**
114k 19 202 220

- ▲ Thanks for this great example. As you point out, some smoothing would be required. Right now we just have two piecewise linear functions that are 'mirrored' about the halfway point. I guess the ideal scenario would be to have two non-linear functions also mirrored about the halfway point and intersecting close to zero?
– [Dipole](#) Sep 4, 2014 at 16:02

▲ Yes, something like that. @jcoppens has a more refined example. – [Warren Weckesser](#) Sep 4, 2014 at 16:21

▲

-3

▼

🔒

🕒

 **This post is hidden.** It was flagged as spam or offensive content and [deleted](#) 3 years ago by [miken32](#), [Community](#) Bot.

 **Locked.** This answer is not currently accepting new interactions. [Learn more.](#)

This answer was marked as spam or rude or abusive and is therefore not shown - you can see [the revision history](#) for details.

Share Edit Follow Flag

answered Sep 3, 2021 at 18:30
user16825383

Comments disabled on deleted / locked posts / reviews